

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Rückblick

Zusammenfassung.

- Felder werden wie folgt deklariert.

```
typ name[anzahl];
```

Der Ausdruck *anzahl* muss einen ganzzahligen Typ und einen positiven Wert haben.

Bei ANSI-C muss es sogar ein konstanter Ausdruck sein

- Die *Komponenten (Elemente)* eines Feldes sind Variablen mit dem Typ des Feldes.

```
name[index]
```

Da die Numerierung bei 0 beginnt, ist der größte zulässige *Index* um 1 kleiner als die Anzahl der Komponenten. Die Indizes selbst können beliebige Ausdrücke mit einem ganzzahligen Typ sein.

- Der Programmierer ist selbst dafür verantwortlich, dass er nur zulässige Indizes verwendet.
- Vektoren mit dem Typ `char` werden als Strings bezeichnet, wenn sie in einer Komponente ein Null-Zeichen enthalten.

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Klassische Felder

Neben eindimensionalen Feldern (**Vektoren**) können auch Felder mit mehreren Dimensionen vereinbart werden.

Dem Feldnamen folgen dann, jeweils in eckigen Klammern eingeschlossen, die Längenangaben für die verschiedenen Dimensionen.

```
int m[2][3];
```

Ergibt ein zweidimensionales Feld (**Matrix**). In gleicher Weise werden mehrere Indizes angegeben, wenn man auf ein einzelnes Element zugreifen will.

```
m[0][0], m[0][1], m[0][2], m[1][0], m[1][1], m[1][2].
```

Felder variabler Größe (*Variable Length Arrays*)

Seit C99 sind lokale **Felder variabler Größe** möglich.

Die Feldgröße muss beim Compilieren nicht feststehen, sondern erst dann, wenn zur Laufzeit lokal, d.h. im Rumpf der zugehörigen Funktion, durch die Deklaration, Speicher für das Feld bereitgestellt wird.

Beispiel

Einlesen der Dimension einer Matrix, dann Einlesen und Ausgeben einer Matrix mit genau dieser Dimension.

Beispiel

```
1  #include <stdio.h>
2
3  int main() {
4      int n, m;
5
6      printf("dimension (nxm): ");
7      scanf("%dx%d", &n, &m);           // e.g. 5x3
8
9      int matrix[n][m];                 // matrix with variable size
10     for (int i = 0; i < n; i++) {      // read matrix
11         for (int j = 0; j < m; j++) {
12             printf("matrix[%d][%d]: ", i, j);
13             scanf("%d", &matrix[i][j]);
14         }
15     }
16
17     for (int i = 0; i < n; i++) {      // write matrix
18         for (int j = 0; j < m; j++)
19             printf("%d ", matrix[i][j]);
20         printf("\n");
21     }
22
23     return 0;
24 }
```

Abstraktion

Sowohl die Vereinbarung eines mehrdimensionalen Feldes als auch die Zugriffe auf seine Komponenten kann man auch in anderer Weise interpretieren.

Eine Matrix zum Beispiel kann man auch als Vektor betrachten, dessen Komponenten ihrerseits Vektoren sind.

Bei einem dreidimensionalen Feld hat man sogar drei Möglichkeiten der Interpretation

- Vektor von Matrizen,
- Matrix von Vektoren,
- Vektor von Vektoren von Vektoren.

Sei `m` als `int m[2][3]` ; deklariert.

`m[0]` und `m[1]` bezeichnet in diesem Sinne die beiden Vektoren der Länge 3, aus denen `m` besteht.

`m[1][0]` bezeichnen so die erste Komponente im zweiten Vektor von `m`.

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Anordnung von Feldern im Speicher

Es ist wichtig zu wissen, wie Felder im Speicher angeordnet sind.

Grundsätzlich wird für jedes Feld ein zusammenhängender Speicherbereich verwendet.

Bei Vektoren sind die Komponenten mit aufsteigenden Indizes hintereinander angeordnet.

Den Index einer Komponente kann man als ihren Abstand vom ersten Element des Vektors interpretieren.

Beispiel

```
int v[6];
```

Speicheranordnung dieses Vektors mit 6 Komponenten.

v[0]	v[1]	v[2]	v[3]	v[4]	v[5]
??	??	??	??	??	??

Matrizen

Die Beschreibung einer Matrix als Vektor von Vektoren legt schon nahe, wie Matrizen im Speicher angeordnet werden.

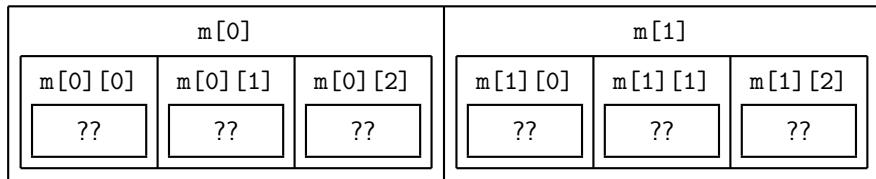
Am Anfang steht der erste Zeilenvektor, dann der zweite Zeilenvektor, usw.

Die Elemente der einzelnen Zeilenvektoren folgen jeweils unmittelbar hintereinander.

Beispiel

```
int m[2][3];
```

Speicheranordnung dieser (2×3) -Matrix.



Speicherabbildungsfunktion

Für den Zugriff auf eine Komponente einer Matrix muss der Abstand vom ersten Element jetzt wirklich *ausgerechnet* werden.

Zugriff $m[i][j]$, dann ist der Abstand $i * 3 + j$.

Allgemeiner Abstand. Zeilenindex mal Zeilenlänge plus Spaltenindex.

Diesen Ausdruck setzt der Compiler bei jedem Zugriff auf eine Matrixkomponente ein.

Die **Speicherabbildungsfunktion** ist in dieser Situation folgende Zuordnung.

$$(i, j) \mapsto i * 3 + j$$

Die Gestalt der Speicherabbildungsfunktion hängt offenbar von der Dimensionierung der Matrix ab. Ähnliche Formeln kann man sich auch für Felder mit höherer Dimension überlegen.

Dass jede Feldkomponente möglicherweise mehrere Bytes belegt und deshalb der Abstand noch einmal mit dieser Anzahl multipliziert werden muss, wird vom Compiler berücksichtigt.

Indexüberschreitung

Auch wenn man mit **unzulässigen** Indizes auf ein Feld zugreift, wird der Abstand ausgerechnet und zum Zugriff verwendet.

Z.B. bewirkt `v[-1]` den Zugriff auf den Speicherbereich direkt **vor** dem Beginn des Vektors. Versuchte Zugriffe auf Feldkomponenten jenseits der Feldgrenzen nennt man **Indexüberschreitung**.

Welche Folgen eine Indexüberschreitung hat, hängt von der Situation ab. Manche bleiben, von eventuell verfälschten Resultaten abgesehen, ohne Wirkung.

Beispiel

```
m[0][3] = m[1][0];
```

Bewirkt nichts.

Das Indexpaar `[0][3]` ist zwar nicht zulässig, da es aber denselben Abstand vom Anfang der Matrix liefert wie das Indexpaar `[1][0]`, nämlich 3, wird nur ein **int**-Wert mit sich selbst überschrieben.

Nicht jeder Verstoß endet so ohne Folgen. Es ist besser wenn ein Programm wegen Zugriffs auf einen *verbotenen* Speicherbereich *gekillt* wird, als wenn es mit verfälschten Daten weiterrechnet.

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Felder als Parameter (1/3)

Eine Funktion kann die Länge eines übergebenen Feldes nicht bestimmen.

Wenn eine Funktion die Länge eines als Argument übergebenen Feldes benötigt, so muss ihr diese mit Hilfe eines separaten Parameters mitgeteilt werden.

Beispiel

Eine Funktion, die die Summe der Komponenten eines Vektors berechnet.

```
1  double vector_sum(const double v[], int length)
2  {
3      double s = 0;
4      while (--length >= 0)
5          s += v[length];
6      return s;
7  }
```

Der Vorteil dieses Vorgehens ist, dass die Funktion für Vektoren beliebiger Länge eingesetzt werden kann.

Der Funktion wird mitgeteilt, wo der Vektor beginnt (`v[]`). Sie unterstellt, dass weitere `length - 1` Komponenten folgen (Verantwortung des Programmierers).

Felder als Parameter (2/3)

Zulässige Aufrufe.

```
double v1[7], v2[50], m[5][20], sum;  
...  
sum = vector_sum(v1, 7) + vector_sum(v2, 50);  
sum = vector_sum(m[3], 20);
```

Nicht zulässige Aufrufe.

```
sum = vector_sum(v1, 10);
```

Die Funktion würde auf Speicher zugreifen, der nicht zu `v1` gehört.

Es steht es dem Benutzer frei, einen Teil der Komponenten von `v2` zu ignorieren.

```
sum = vector_sum(v2, 10);  
sum = vector_sum(&v2[10], 10);
```


Felder als Parameter (3/3)

Folgendes ist zulässig.

```
double v1[7], v2[50], m[5][20], sum;  
...  
sum = vector_sum(m[0], 100);
```

Die Matrix `m` besteht aus insgesamt 100 Komponenten, die im Speicher unmittelbar aufeinanderfolgen. Diesen Speicherbereich kann bei Bedarf auch als Vektor betrachtet werden.

Aber nur `m` als Parameter anzugeben reicht nicht aus. `m` bezeichnet einen **Vektor von Zeilenvektoren**, nicht einen Vektor von `double`-Werten, deshalb `m[0]`.

Felder variable Größe als Parameter

Felder variable Größe sind auch als Parameter von Funktionen zulässig (C99).

Die Länge des Feldes kann über einen anderen Parameter der Funktion bestimmt werden, der aber **vor** dem Parameter für das Feld variabler Größe stehen muss.

```
1 double vector_sum2(int length, double v[length])
2 {
3     double s = 0;
4     while (--length >= 0)
5         s += v[length];
6     return s;
7 }
```

In diesem Beispiel macht das, bis auf die Reihenfolge der Parameter, keine Unterschied zur klassischen Realisierung, der Unterschied wird erst bei mehrdimensionalen Feldern als Parameter sichtbar.

Mehrdimensionale Feld als Parameter (1/3)

Wenn ein mehrdimensionales Feld in die Funktion als solches betrachtet werden soll und wenn obendrein die Größe des Feldes in den einzelnen Dimensionen von Aufruf zu Aufruf verschieden sein kann, wird es schwieriger.

Mehrdimensionale Feld als Parameter (2/3)

In einem Funktionsheader **müssen** die Längenangaben aller Dimensionen, ausser der erste Dimension (diese wird ignoriert), angegeben werden.

Der Prototyp

```
double f(double m[][3], int rows);
```

ist zulässig, erlaubt den Einsatz der Funktion `f` aber nur für Matrizen mit 3 Spalten und ist deshalb für die Praxis unzureichend.

Folgender Prototyp ist **nicht** erlaubt.

```
double f(double m[][], int rows, int columns);
```

Ohne die Längenangabe der Zeilen hat der Compiler keine Möglichkeit, die Speicherabbildungsfunktion zu berechnen.

Eine mögliche Lösung sind **Felder variabler Größe**.

```
double f(int rows, int columns, double m[rows][columns]);
```

Mehrdimensionale Feld als Parameter (3/3)

Klassische Lösung

Man übergibt die Matrix formal als Vektor und gibt diesem Vektor in der Funktion wieder Matrixstruktur, indem man die Speicherabbildungsfunktion selbst berechnet.

Zu Übungszwecken wird diese Lösung hier auch vorgestellt werden.

Beispiel

Matrizenprodukt $C = A \cdot B$ für Matrizen $A = (a_{ij})$, $B = (b_{jk})$ und $C = (c_{ik})$ mit $i = 1, \dots, \ell$, $j = 1, \dots, m$ und $k = 1, \dots, n$.

$$c_{ik} = \sum_{j=1}^m a_{ij} \cdot b_{jk}$$

Prototyp einer entsprechenden Funktion.

```
1 void matprod(double c[], const double a[], const double b[],
2             int l, int m, int n);
```

Beispiel Matrixprodukt (1/3)

Die Speicherabbildungsfunktion um den übergebenen Vektoren wieder Matrixstruktur zu geben, wird als Makro (Präprozessoranweisung) definiert.

Der Makro berechnet zu jedem Indexpaar den Abstand vom Anfang des Vektors, dazu wird zusätzlich zu den beiden Indizes auch die Länge der Zeilen der Matrix als Parameter benötigt.

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Makros

Mit der Direktive `#define` kann man nicht nur Konstanten benennen.

Form der Direktive.

```
#define name [ersatztext]
```

Namen, die in einer `define`-Direktive definiert werden, werden auch als **Makros** bezeichnet.

Die Ersetzungen, die ein Makro bewirkt, bezeichnet man als **Expandierung des Makro**.

Als *ersatztext* sind beliebige Zeichenfolgen erlaubt. Diese Zeichenfolgen dürfen insbesondere auch bereits zuvor definierte Makros enthalten oder leer sein.

Beispiel

```
#define WIDTH 10  
#define HEIGHT (WIDTH + 5)
```


Makros mit Parameter (1/2)

Makros können Parameter erhalten.

```
#define name(parameter) ersatztext
```

Wichtig ist, dass die öffnende Klammer dem Namen **unmittelbar** folgen muss damit wird gekennzeichnet, dass die Klammer eine Parameterliste einleitet und nicht Bestandteil des Ersatztextes ist.

Beispiel

```
#define SQUARE(x) x * x
```

An den Stellen, an denen der Makro expandiert werden soll, müssen Argumente angegeben werden, ähnlich wie bei einer Funktion.

Makros mit Parameter (2/2)

Makros mit Parameter führen praktisch zu einer **doppelte** Textersetzung.

Der Makro einschließlich seiner Argumentliste wird durch den Ersatztext ersetzt; im Ersatztext werden die Parameter durch die Argumente ersetzt.

Beispiel

```
x = SQUARE(7.4);  
y = SQUARE(x + 1);
```

Ergibt letztlich folgende Quellanweisungen.

```
x = 7.4 * 7.4;  
y = x + 1 * x + 1;
```

Der Makro **SQUARE** ist **unvollständig** definiert. Für **y** resultiert ein Ausdruck, der **nicht** der Intention entspricht.

Abhilfe

```
#define SQUARE(x) ((x) * (x))
```

Die äußeren Klammern sind nötig, da es Operatoren gibt, deren Priorität höher als die der Multiplikation ist.

Makros / Funktionen

Makros mit Parametern können wie Funktionen die Lesbarkeit eines Programms erhöhen und Schreibarbeit sparen.

Zwischen Makros und Funktionen bestehen aber ganz wesentliche Unterschiede, die letztlich alle aus der Tatsache resultieren, dass Makros expandiert werden, **bevor** die eigentliche Übersetzung beginnt und dass sie damit im ausführbaren Programm überhaupt nicht mehr in Erscheinung treten.

- Die Parameter von Makros besitzen **keine** Typen, so dass die Argumente beliebige Typen besitzen können.

Typen spielen erst eine Rolle, wenn der Compiler übersetzt.

- Jedes Vorkommen eines Makro wird durch den Ersatztext ersetzt.

Das hat zur Folge, dass der Maschinencode, der aus dem Ersatztext resultiert, mehrfach im ausführbaren Programm steht, während der Maschinencode, der aus einer Funktion resultiert, nur einmal im Programm steht.

Dadurch entfallen die Sprünge und die Parameterzuordnung, mit denen Funktionsaufrufe verbunden sind.

Makros und Nebeneffekte

Aus der Tatsache, dass ein Makro-Aufruf durch Textersetzung aufgelöst wird, folgt auch, dass man vorsichtig sein muss, wenn man als Argumente Ausdrücke mit Nebeneffekten einsetzt.

```
i = SQUARE(j++);
```

Wird expandiert zu

```
i = ((j++) * (j++));
```

Abgesehen davon, dass der doppelte Nebeneffekt in der Regel nicht beabsichtigt sein wird, ist er auch gefährlich.

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Beispiel Matrixprodukt klassisch (1/2)

Die Speicherabbildungsfunktion um den übergebenen Vektoren wieder Matrixstruktur zu geben, wird als Makro (Präprozessoranweisung) definiert.

Der Makro berechnet zu jedem Indexpaar den Abstand vom Anfang des Vektors, dazu wird zusätzlich zu den beiden Indizes auch die Länge der Zeilen der Matrix als Parameter benötigt.

In diesen Makro kann dann auch gleich noch die Transformation der logischen Indizes $1, \dots, x$ in die C-Indizes $0, \dots, x - 1$ hineingesteckt werden.

```
#define lin(row, column, length) \  
    (((row)-1)*(length))+((column)-1))
```

Beispiel Matrixprodukt klassisch (2/2)

Definition der Funktion in einer Schreibweise, die der Formel zumindest einigermaßen nahekkommt.

```
1  #define lin(row, column, length)          \
2      (((row)-1)*(length)+(column)-1)
3
4
5  void matprod1(double c[], const double a[], const double b[],
6      int l, int m, int n)
7  {
8      int i, j, k;
9      for (i = 1; i <= l; i++)
10         for (k = 1; k <= n; k++) {
11             c[lin(i,k,n)] = 0;
12             for (j = 1; j <= m; j++)
13                 c[lin(i,k,n)] += a[lin(i,j,m)] * b[lin(j,k,n)];
14         }
15 }
```

Beispiel Matrixprodukt Aufruf

```
1  #include <stdio.h>
2
3  int main() {
4      int l, m, n;
5
6      printf("Dimensionen (l,m,n): ");
7      scanf("%d,%d,%d", &l, &m, &n);    // z.B. 5,3,7
8
9      double a[l][m];                    // Matrizen variabler Groesse
10     double b[m][n];
11     for (int i = 0; i < l; i++) {       // Einlesen von a
12         for (int j = 0; j < m; j++) {
13             printf("a[%d][%d]: ", i, j);
14             scanf("%lf", &a[i][j]);
15         }
16     }
17     ...                                // Einlesen von b
18
19     double c[l][n];                    // Matrix variabler Groesse
20     matprod1(&c[0][0], a[0], (double *)b, l, m, n);
21
22     ...                                // Arbeiten mit c
23     return 0;
24 }
```


Beispiel Matrixprodukt mit Felder variable Größe (1/2)

Übergebene Felder werden lokal mit Struktur einer Matrix versehen.

```
1 void matprod2(double c[], const double a[], const double b[],
2               int l, int m, int n)
3 {
4     double (*aa)[m] = (double(*)[m]) a;
5     double (*bb)[n] = (double(*)[n]) b;
6     double (*cc)[n] = (double(*)[n]) c;
7
8     for (int i = 0; i < l; i++)
9         for (int k = 0; k < n; k++) {
10             cc[i][k] = 0;
11             for (int j = 0; j < m; j++)
12                 cc[i][k] += aa[i][j] * bb[j][k];
13         }
14 }
```

Argumente für den Funktionsaufruf bleiben gleich.

Bemerkung

In dieser Version ist gut zu sehen, dass ein Zeiger auf einen Vektor wie ein Feld von Vektoren funktioniert. Die Adresse von `aa[i]` wird bestimmt, indem `i`-mal die Länge eines **double-Feldes mit `m` Komponenten** zur Adresse von `aa` addiert wird.

Beispiel Matrixprodukt mit Felder variable Größe (2/2)

Parameter sind Felder variabler Größe.

```
1 void matprod3(int l, int m, int n,  
2             double a[l][m], double b[m][n], double c[l][n])  
3 {  
4     for (int i = 0; i < l; i++)  
5         for (int k = 0; k < n; k++) {  
6             c[i][k] = 0;  
7             for (int j = 0; j < m; j++)  
8                 c[i][k] += a[i][j] * b[j][k];  
9         }  
10 }
```

Funktionsaufruf muss jetzt entsprechend angepasst werden.

```
1 double a[l][m];           // Matrizen variabler Groesse  
2 double b[m][n];  
3 ...                       // Initialisieren von a, b  
4 double c[l][n];           // Matrix variabler Groesse  
5 matprod3(l, m, n, a, b, c);  
6 ...                       // Arbeiten mit c
```

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Initialisierung von Feldern

Felder können initialisiert werden.

Für Strings haben wurde diese Möglichkeit bereits genutzt, aber die allgemeine Initialisierung von Felder sieht anders aus.

Anfangswertzuweisung für Felder. Die Werte der Komponenten werden einzeln aufgeführt, voneinander jeweils durch ein Komma getrennt und insgesamt in geschweifte Klammern eingeschlossen.

```
int v[6] = {6, 5, 4, 17, 18, 19};
```

Gibt man weniger Werte als die Dimension des Feldes an, werden die übrigen auf Null gesetzt. Gibt man zu viele Werte an, wird der Compiler einen Fehler anzeigen.

Wenn man für jede Komponente einen Wert angibt, kann man auch den Compiler abzählen lassen und die Feldlänge ganz weglassen.

Folgende Definition ist zulässig und ergibt einen Vektor mit 6 Komponenten.

```
int v[] = {6, 5, 4, 17, 18, 19};
```

Initialisierung von mehrdimensionalen Feldern

Bei der Initialisierung mehrdimensionaler Felder muss man sich wieder klar machen, dass sie formal Vektoren von Vektoren sind.

Beispiel

```
int m[2][3] = {{ 6,  5,  4},  
               {17, 18, 19}};
```

Wenn man Anfangswerte angibt, darf man bei mehrdimensionalen Feldern die Länge in der ersten Dimension weglassen; der Compiler bestimmt sie dann durch Abzählen.

Das ist nicht nur für Schreibfaule interessant, sondern erspart einem einerseits das Abzählen der Daten und verhindert andererseits ein Verzählen dabei.

Teilweise Initialisierung von mehrdimensionalen Feldern

Auch mehrdimensionale Felder lassen sich teilweise initialisieren.

Alle nicht angegebenen Komponenten erhalten den Wert 0.

Die durch die geschweiften Klammern angegebene Struktur wird eingehalten.

```
double v[12] = { 0 };    /* alle Komponenten = 0 */

int m[4][2][3] = {{{ 1, 2, 3 }, { 4, 5, 6 }},
                  {{ 7, 8 }},
                  {{ 9 }, { 10 }}}};

/* entspricht = {{{ 1, 2, 3 }, { 4, 5, 6 }},
                  {{ 7, 8, 0 }, { 0, 0, 0 }},
                  {{ 9, 0, 0 }, { 10, 0, 0 }},
                  {{ 0, 0, 0 }, { 0, 0, 0 }}} */
```

Man hätte die erste Dimension des Feldes `m` weglassen können. Der Compiler hätte für sie dann 3 bestimmt, da für drei Zeilen Anfangswerte angegeben wurden.

Initialisierung von Strings

Für Strings gibt es eine Kurzform der Initialisierung

Wenn eine Stringkonstante angegeben ist, zerlegt der Compiler diese in einzelne Zeichen und sorgt selber für das abschließende Null-Zeichen.

Folgende Vereinbarungen sind äquivalent.

```
char str1[6] = "hallo",  
    str2[6] = {'h', 'a', 'l', 'l', 'o', '\0'};
```

Da man auch hier den Compiler abzählen lassen kann, sind die folgenden Vereinbarungen auch äquivalent.

```
char str3[] = "hallo",  
    str4[] = {'h', 'a', 'l', 'l', 'o', '\0'};
```

Alle Konstanten, die in den Initialisierungen angegeben wurden, dürften auch konstante Ausdrücke sein.

Felder

Rückblick

Vereinbarung von Feldern

Anordnung von Feldern im Speicher

Felder als Parameter I

Einschub: Makros

Felder als Parameter II

Initialisierung von Feldern

Zeiger und Felder

Zeiger und Felder

Anfangswerte zuweisen.

```
char str1[6] = "hallo",  
    str2[6] = {'h', 'a', 'l', 'l', 'o', '\0'},  
    str3[]   = "hallo",  
    str4[]   = {'h', 'a', 'l', 'l', 'o', '\0'};
```

Da nur Anfangswerte festgelegt werden, ist folgendes ohne weiteres erlaubt.

```
str1[1] = 'o';  
str1[4] = 'a';
```

Der Wert wird zu "holla".

Eine andere Wirkung hat die ebenfalls zulässige Anfangswertzuweisung.

```
char *str5 = "hallo";
```

`str5` bezeichnet nicht mehr einen Vektor, in dem der String gespeichert wird, sondern eine Variable, deren Anfangswert der Zeiger auf den Anfang des Strings ist.

Verändern des Zeigers

Die Wertzuweisung im folgenden Beispiel ist **unzulässig**, weil im Feld `str` nur `char`-Werte und nicht Zeiger (auf Konstanten) gespeichert werden können.

```
str1[1] = 'o';  
char *str5 = "hallo";
```

```
str1 = str5;
```

Zulässig ist aber den Wert der Zeigervariablen zu verändern.

```
str5 = str1;
```

Ob das allerdings vernünftig ist, ist eine andere Frage.

Wenn der Wert von `str5` verändert wird, ohne dass den Anfangswert zuvor in eine andere Zeigervariable umspeichern, hat man in der Folge **keinen Zugriff** mehr auf den String (auch wenn er natürlich unverändert im Speicher steht). Oft ist es zweckmäßig solche Zeigervariablen mit dem Attribut `const` zu versehen.

```
char *const str5 = "hallo";
```

Jetzt darf der Wert von `str5` im Programm nicht mehr verändert werden.

Zuweisung von Stringkonstanten

Was bedeutet bzw. bewirkt folgende Wertzuweisung?

```
s = "hallo";
```

C kennt keine Wertzuweisung, bei der der Wert eines String (der Wert eines Feldes) übertragen wird.

Der String wird faktisch als Zeiger auf den Anfang der Zeichenfolge interpretiert, wobei die Zeichenfolge selbst vom Compiler irgendwo im Speicher (üblicherweise im Datensegment) abgelegt wird.

Die Variable **s** **muss** damit den Typ `char *` besitzen.

Zulässig ist also Folgendes.

```
char *s;
```

```
s = "hallo";
```